



PREMIER UNIVERSITY CHATTOGRAM

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

LAB REPORT

COURSE NAME	Big Data Analytics Lab	
COURSE CODE	CSE 4346	
REPORT NAME	HOTEL BOOKING DATA ANALYSIS USING HADOOP MAPREDUCE AND APACHE PIG	
DATE OF ASSIGNMENT	01-09-2026	
DATE OF SUBMISSION	15-09-2026	
SUBMITTED TO		
MD Tamim Hossain Lecturer, Department of CSE, Premier University		
REMARKS	SUBMITTED BY	
	NAME	MD. Tarek Hossen
	ID	0222220005101004
	SEMESTER	8th
	BATCH	42nd
	SESSION	Spring 2026
	SECTION	A

Table of Contents

Serial No	Topic	Page No
1	Problem Statement	3
2	Dataset Description	3
3	Summary of Data Analysis	4
4	Environment and Dataset Preparation	4
5	MapReduce Analysis	8
5.1	Top 10 Countries by Number of Bookings	8
5.2	Cancellations by Market Segment	10
5.3	Top 10 Countries by Total Stayed Nights	13
6	Pig Hotel Booking Data Analysis	15
6.1	Top 5 Countries by Booking Count	15
6.2	Top 10 Countries by Average ADR	17
6.3	Top 10 Countries by Number of Cancellations	19
6.4	Market Segments by Booking Count	21
6.5	Top 10 Arrival Months by Booking Count	23
7	GitHub Repository	25
8	Conclusion	26
9	References	26

1. Problem Statement

Analyze the Hotel Booking Demand dataset using Hadoop MapReduce and Apache Pig in order to produce useful summaries, rankings, cancellation patterns, pricing insights, market-segment analysis, and arrival-month trends. The implementation uses Hadoop Distributed File System (HDFS) for storage, Java MapReduce for distributed processing, and Apache Pig for high-level data analysis. The project implements three MapReduce analyses and five Pig analyses on the same cleaned dataset.

2. Dataset Description

Dataset source: <https://www.kaggle.com/datasets/jessemostipak/hotel-booking-demand>

Raw file: hotel_bookings.csv. The file contains 119,390 booking records plus one header line (119,391 lines in total), 32 original columns, and a size of 16,855,599 bytes. The dataset contains booking information for Resort Hotel and City Hotel records, including cancellation status, lead time, arrival period, stay duration, guest origin, market segment, booking channel, customer type, Average Daily Rate (ADR), and reservation status.

Field	Description
hotel	Hotel type: Resort Hotel or City Hotel.
is_canceled	1 if the booking was canceled; 0 otherwise.
lead_time	Number of days between booking entry and arrival date.
arrival_date_year	Year of arrival.
arrival_date_month	Month of arrival.
arrival_date_week_number	Week number of arrival within the year.
arrival_date_day_of_month	Day of month of arrival.
stays_in_weekend_nights	Number of weekend nights in the booking.
stays_in_week_nights	Number of weekday nights in the booking.
adults	Number of adults.
children	Number of children.
babies	Number of babies.
meal	Booked meal type.
country	Guest country/origin code.
market_segment	Market segment such as Online TA, Direct, Groups, etc.
distribution_channel	Booking distribution channel.
is_repeated_guest	1 for repeated guest; 0 otherwise.
previous_cancellations	Previous cancellations by the customer.
previous_bookings_not_canceled	Previous bookings not canceled.
reserved_room_type	Reserved room-type code.
assigned_room_type	Assigned room-type code.
booking_changes	Number of booking changes.
deposit_type	Deposit policy/type.
agent	Travel-agent identifier where available.

company	Company identifier where available.
days_in_waiting_list	Days spent on the waiting list.
customer_type	Customer type, e.g., Transient or Contract.
adr	Average Daily Rate for the booking.
required_car_parking_spaces	Number of required parking spaces.
total_of_special_requests	Number of special requests.
reservation_status	Reservation status such as Check-Out or Canceled.
reservation_status_date	Date of final reservation status.

3. Summary of Data Analysis

The following MapReduce analyses were implemented:

1. Top 10 Countries by Number of Bookings.
2. Cancellations by Market Segment.
3. Top 10 Countries by Total Stayed Nights (non-canceled bookings).

The following Pig analyses were implemented:

4. Top 5 Countries by Booking Count.
5. Top 10 Countries by Average ADR.
6. Top 10 Countries by Number of Cancellations.
7. Market Segments by Booking Count.
8. Top 10 Arrival Months by Booking Count.

The principal processing pattern is summarization and ranking. MapReduce Mapper classes extract keys and measures from valid TSV records, Hadoop performs shuffle/grouping, and Reducer classes aggregate values. Secondary jobs are used where a global Top 10 or descending ranking is required. Pig scripts use LOAD, FILTER, GROUP, FOREACH, ORDER, LIMIT, AVG, COUNT and STORE operations.

4. Environment and Dataset Preparation

Hadoop Environment

Environment used: Windows 10, Java 1.8.0_411, Hadoop 3.2.4 and Apache Pig 0.18.0. Hadoop NameNode, DataNode, ResourceManager and NodeManager were verified through JPS.

```
C:\hadoop>start-dfs.cmd

C:\hadoop>jps
12352 NodeManager
15744 ResourceManager
15972 DataNode
11548 NameNode
16028 Jps
```

Hadoop daemons confirmed through JPS.

Project Directory Structure

The project was organized into separate folders for raw/processed data, Java preprocessing, three MapReduce analyses, Pig scripts and outputs, and screenshots.

```
C:\BDALFinalProject>tree /F
Folder PATH listing for volume SSD
Volume serial number is 16B2-D78D
C:..
|---CancellationByMarketSegment
|---data
|   |---processed
|   |---raw
|       hotel_bookings.csv
|---PigAnalysis
|   |---outputFiles
|   |---scripts
|---Preprocessing
|---screenshots
|---Top10Countries
|---Top10CountriesByStayedNights
```

Project directory structure.

Raw Dataset Verification

The raw CSV file was stored under `data\raw\hotel_bookings.csv`. The original file size was 16,855,599 bytes. A command-line line count returned 119,391 lines, corresponding to 119,390 records plus the header.

```
dir hotel_bookings* -> 16,855,599 bytes
find /v /c "" hotel_bookings.csv -> 119391 lines
```

Java Preprocessing

A Java preprocessing program based on Apache Commons CSV parsed the raw CSV safely, selected 17 fields needed by the analyses, removed tab/newline characters from text fields, validated numeric fields, and converted missing country values to UNKNOWN. This avoided unsafe comma splitting and produced a tab-separated input suitable for both MapReduce and Pig.

```
import org.apache.commons.csv.CSVFormat;
import org.apache.commons.csv.CSVParser;
import org.apache.commons.csv.CSVRecord;

import java.io.BufferedWriter;
import java.io.Reader;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Paths;

public class HotelBookingPreprocessor {
    private static String clean(String value) {
        if (value == null) return "";
        return value.replace("\t", " ")
            .replace("\r", " ")
            .replace("\n", " ")
            .trim();
    }

    public static void main(String[] args) {
        if (args.length != 2) {
            System.out.println("Usage: HotelBookingPreprocessor <input.csv> <output.tsv>");
            return;
        }

        String inputFile = args[0];
        String outputFile = args[1];
        long totalRows = 0, validRows = 0, skippedRows = 0, missingCountry = 0;

        try (Reader reader = Files.newBufferedReader(Paths.get(inputFile), StandardCharsets.UTF_8);
            CSVParser parser = CSVFormat.DEFAULT.withFirstRecordAsHeader()
                .withIgnoreHeaderCase().withTrim().parse(reader);
            BufferedWriter writer = Files.newBufferedWriter(Paths.get(outputFile), StandardCharsets.UTF_8)) {

            for (CSVRecord record : parser) {
                totalRows++;
                try {
                    String hotel = clean(record.get("hotel"));
                }
            }
        }
    }
}
```

```

String isCanceled = clean(record.get("is_canceled"));
String leadTime = clean(record.get("lead_time"));
String arrivalYear = clean(record.get("arrival_date_year"));
String arrivalMonth = clean(record.get("arrival_date_month"));
String weekendNights = clean(record.get("stays_in_weekend_nights"));
String weekNights = clean(record.get("stays_in_week_nights"));
String adults = clean(record.get("adults"));
String country = clean(record.get("country"));
String marketSegment = clean(record.get("market_segment"));
String distributionChannel = clean(record.get("distribution_channel"));
String repeatedGuest = clean(record.get("is_repeated_guest"));
String depositType = clean(record.get("deposit_type"));
String customerType = clean(record.get("customer_type"));
String adr = clean(record.get("adr"));
String reservationStatus = clean(record.get("reservation_status"));
String reservationStatusDate = clean(record.get("reservation_status_date"));

Integer.parseInt(isCanceled);
Integer.parseInt(leadTime);
Integer.parseInt(arrivalYear);
Integer.parseInt(weekendNights);
Integer.parseInt(weekNights);
Integer.parseInt(adults);
Integer.parseInt(repeatedGuest);
Double.parseDouble(adr);

if (country.isEmpty() || country.equalsIgnoreCase("NULL")) {
    country = "UNKNOWN";
    missingCountry++;
}

writer.write(hotel + "\t" + isCanceled + "\t" + leadTime + "\t" +
    arrivalYear + "\t" + arrivalMonth + "\t" + weekendNights + "\t" +
    weekNights + "\t" + adults + "\t" + country + "\t" + marketSegment +
    "\t" + distributionChannel + "\t" + repeatedGuest + "\t" + depositType +
    "\t" + customerType + "\t" + adr + "\t" + reservationStatus + "\t" +
    reservationStatusDate);
writer.newLine();
validRows++;
} catch (Exception e) {
    skippedRows++;
}
}

System.out.println("HOTEL BOOKING PREPROCESSING COMPLETE");
System.out.println("Total raw rows : " + totalRows);
System.out.println("Valid rows : " + validRows);
System.out.println("Skipped rows : " + skippedRows);
System.out.println("Missing country : " + missingCountry);
System.out.println("Output file : " + outputFile);
} catch (Exception e) {
    System.out.println("Preprocessing failed.");
    e.printStackTrace();
}
}
}
}

```

Preprocessing command:

```

mvn exec:java -Dexec.args="C:\BDALFinalProject\data\raw\hotel_bookings.csv
C:\BDALFinalProject\data\processed\Hotel_Booking_Clean.tsv"

```

```

=====
HOTEL BOOKING PREPROCESSING COMPLETE
=====
Total raw rows : 119390
Valid rows : 119390
Skipped rows : 0
Missing country : 488
Output file : C:\BDALFinalProject\data\processed\Hotel_Booking_Clean.tsv
=====
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 3.491 s
[INFO] Finished at: 2026-09-15T19:35:17+06:00
[INFO] -----

```

Java preprocessing completed successfully.

Preprocessing summary: 119,390 total raw rows, 119,390 valid rows, 0 skipped rows, and 488 missing-country values replaced with UNKNOWN. The clean TSV contained 119,390 rows and had a size of 12,588,282 bytes.

Clean TSV Field Order

Index	Field
0	hotel
1	is_canceled
2	lead_time
3	arrival_date_year
4	arrival_date_month
5	stays_in_weekend_nights
6	stays_in_week_nights
7	adults
8	country
9	market_segment
10	distribution_channel
11	is_repeated_guest
12	deposit_type
13	customer_type
14	adr
15	reservation_status
16	reservation_status_date

```
C:\BDALFinalProject\Preprocessing>more C:\BDALFinalProject\data\processed\Hotel_Booking_Clean.tsv
Resort Hotel 0 342 2015 July 0 0 2 PRT Direct Direct 0 No Deposit Transient
0 Check-Out 2015-07-01
Resort Hotel 0 737 2015 July 0 0 2 PRT Direct Direct 0 No Deposit Transient
0 Check-Out 2015-07-01
Resort Hotel 0 7 2015 July 0 1 1 GBR Direct Direct 0 No Deposit Transient
75 Check-Out 2015-07-02
Resort Hotel 0 13 2015 July 0 1 1 GBR Corporate Corporate 0 No Deposit
Transient 75 Check-Out 2015-07-02
Resort Hotel 0 14 2015 July 0 2 2 GBR Online TA TA/TO 0 No Deposit Tra
nsient 98 Check-Out 2015-07-03
Resort Hotel 0 14 2015 July 0 2 2 GBR Online TA TA/TO 0 No Deposit Tra
nsient 98 Check-Out 2015-07-03
Resort Hotel 0 0 2015 July 0 2 2 PRT Direct Direct 0 No Deposit Transient
107 Check-Out 2015-07-03
Resort Hotel 0 9 2015 July 0 2 2 PRT Direct Direct 0 No Deposit Transient
103 Check-Out 2015-07-03
```

Sample records from the cleaned Hotel Booking TSV dataset.

HDFS Upload

The cleaned TSV file was uploaded to HDFS at /user/DELL/hotel_project/input/Hotel_Booking_Clean.tsv. The HDFS listing reported approximately 12.0 MB.

```
hdfs dfs -put C:\BDALFinalProject\data\processed\Hotel_Booking_Clean.tsv /user/DELL/hotel_project/input/
```

```
C:\hadoop>hdfs dfs -ls -h /user/DELL/hotel_project/input
Found 1 items
-rw-r--r-- 1 DELL supergroup 12.0 M 2026-09-15 19:41 /user/DELL/hotel_project/input/Hotel_Booking_Clean.tsv
```

Cleaned Hotel Booking dataset successfully uploaded to HDFS.

5. MapReduce Analysis

Three Java MapReduce analyses were implemented. Each analysis uses a Driver class to configure the jobs, a Mapper class to emit intermediate key-value pairs, and a Reducer class to aggregate values. Operations that require global ranking use a second job with one reducer.

5.1 Top 10 Countries by Number of Bookings

This analysis counts valid bookings by guest country and then selects the ten countries with the largest booking counts. UNKNOWN countries are excluded. Job 1 performs country-level aggregation; Job 2 produces the global Top 10.

Mapper Class

```
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class Top10CountriesMapper extends Mapper<LongWritable, Text, Text, IntWritable> {
    private static final IntWritable ONE = new IntWritable(1);
    private final Text countryKey = new Text();

    public enum BookingCounters { MALFORMED_ROWS, MISSING_COUNTRY }

    @Override
    protected void map(LongWritable key, Text value, Context context)
        throws IOException, InterruptedException {
        String[] fields = value.toString().split("\\t", -1);
        if (fields.length < 17) {
            context.getCounter(BookingCounters.MALFORMED_ROWS).increment(1);
            return;
        }
        String country = fields[8].trim();
        if (country.isEmpty() || country.equalsIgnoreCase("UNKNOWN")) {
            context.getCounter(BookingCounters.MISSING_COUNTRY).increment(1);
            return;
        }
        countryKey.set(country);
        context.write(countryKey, ONE);
    }
}
```

Reducer Class

```
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class Top10CountriesReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
    @Override
    protected void reduce(Text key, Iterable<IntWritable> values, Context context)
        throws IOException, InterruptedException {
        int totalBookings = 0;
        for (IntWritable value : values) totalBookings += value.get();
        context.write(key, new IntWritable(totalBookings));
    }
}
```

Driver Class

```
import java.io.IOException;
import java.util.*;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class Top10CountriesDriver {
    public static class Top10Mapper extends Mapper<LongWritable, Text, NullWritable, Text> {
        @Override
        protected void map(LongWritable key, Text value, Context context)
            throws IOException, InterruptedException {
```



```

        String line = value.toString().trim();
        if (line.isEmpty()) return;
        String[] parts = line.split("\\t");
        if (parts.length < 2) return;
        try {
            String country = parts[0].trim();
            int count = Integer.parseInt(parts[1].trim());
            context.write(NullWritable.get(), new Text(country + "\\t" + count));
        } catch (NumberFormatException e) { }
    }
}

public static class Top10Reducer extends Reducer<NullWritable, Text, Text, IntWritable> {
    private static class CountryCount {
        String country; int count;
        CountryCount(String country, int count) { this.country = country; this.count = count; }
    }

    @Override
    protected void reduce(NullWritable key, Iterable<Text> values, Context context)
        throws IOException, InterruptedException {
        PriorityQueue<CountryCount> top10 = new PriorityQueue<>(10,
            Comparator.comparingInt(a -> a.count));
        for (Text value : values) {
            String[] parts = value.toString().split("\\t");
            if (parts.length < 2) continue;
            try {
                CountryCount cc = new CountryCount(parts[0].trim(),
                    Integer.parseInt(parts[1].trim()));
                top10.add(cc);
                if (top10.size() > 10) top10.poll();
            } catch (NumberFormatException e) { }
        }
        List<CountryCount> result = new ArrayList<>(top10);
        result.sort((a,b) -> Integer.compare(b.count, a.count));
        for (CountryCount item : result)
            context.write(new Text(item.country), new IntWritable(item.count));
    }
}

public static void main(String[] args) throws Exception {
    if (args.length != 3) {
        System.err.println("Usage: Top10CountriesDriver <input> <intermediate_output> <final_output>");
        System.exit(1);
    }
    Configuration conf = new Configuration();
    Job job1 = Job.getInstance(conf, "Hotel Booking - Country Counts");
    job1.setJarByClass(Top10CountriesDriver.class);
    job1.setMapperClass(Top10CountriesMapper.class);
    job1.setReducerClass(Top10CountriesReducer.class);
    job1.setMapOutputKeyClass(Text.class);
    job1.setMapOutputValueClass(IntWritable.class);
    job1.setOutputKeyClass(Text.class);
    job1.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job1, new Path(args[0]));
    FileOutputFormat.setOutputPath(job1, new Path(args[1]));
    if (!job1.waitForCompletion(true)) System.exit(1);

    Job job2 = Job.getInstance(conf, "Hotel Booking - Top 10 Countries");
    job2.setJarByClass(Top10CountriesDriver.class);
    job2.setMapperClass(Top10Mapper.class);
    job2.setReducerClass(Top10Reducer.class);
    job2.setNumReduceTasks(1);
    job2.setMapOutputKeyClass(NullWritable.class);
    job2.setMapOutputValueClass(Text.class);
    job2.setOutputKeyClass(Text.class);
    job2.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job2, new Path(args[1]));
    FileOutputFormat.setOutputPath(job2, new Path(args[2]));
    System.exit(job2.waitForCompletion(true) ? 0 : 1);
}
}

```

Execution

hadoop jar target\top10-countries-1.0.jar /user/DELL/hotel_project/input/Hotel_Booking_Clean.tsv
/user/DELL/hotel_project/output/country_counts /user/DELL/hotel_project/output/top10_countries

```

2026-09-15 19:51:53,505 INFO mapreduce.Job: Running job: job_1789479626440_0002
2026-09-15 19:52:05,694 INFO mapreduce.Job: Job job_1789479626440_0002 running in uber mode : false
2026-09-15 19:52:05,694 INFO mapreduce.Job: map 0% reduce 0%
2026-09-15 19:52:11,824 INFO mapreduce.Job: map 100% reduce 0%
2026-09-15 19:52:16,884 INFO mapreduce.Job: map 100% reduce 100%
2026-09-15 19:52:16,898 INFO mapreduce.Job: Job job_1789479626440_0002 completed successfully
2026-09-15 19:52:16,935 INFO mapreduce.Job: Counters: 54
File System Counters

```

MR1 execution completed successfully (map 100%, reduce 100%).

Result

```
C:\BDALFinalProject\Top10Countries>hdfs dfs -cat /user/DELL/hotel_project/output/top10_countries/part-r-00000
PRT      48590
GBR      12129
FRA      10415
ESP      8568
DEU      7287
ITA      3766
IRL      3375
BEL      2342
BRA      2224
NLD      2104
```

Top 10 countries by number of bookings.

Country	Bookings
PRT	48,590
GBR	12,129
FRA	10,415
ESP	8,568
DEU	7,287
ITA	3,766
IRL	3,375
BEL	2,342
BRA	2,224
NLD	2,104

Interpretation: Portugal (PRT) had the highest number of bookings with 48,590 records, followed by the United Kingdom (GBR) with 12,129 and France (FRA) with 10,415.

5.2 Cancellations by Market Segment

This analysis filters records where `is_canceled = 1`, groups them by `market_segment`, counts cancellations in each segment, and sorts the final counts in descending order.

Mapper Class

```
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class CancellationByMarketSegmentMapper
    extends Mapper<LongWritable, Text, Text, IntWritable> {
    private static final IntWritable ONE = new IntWritable(1);
    private final Text segmentKey = new Text();
    public enum CancellationCounters { MALFORMED_ROWS, MISSING_MARKET_SEGMENT }

    @Override
    protected void map(LongWritable key, Text value, Context context)
        throws IOException, InterruptedException {
        String[] fields = value.toString().split("\\t", -1);
        if (fields.length < 17) {
            context.getCounter(CancellationCounters.MALFORMED_ROWS).increment(1);
            return;
        }
        String isCanceled = fields[1].trim();
        String marketSegment = fields[9].trim();
        if (!isCanceled.equals("1")) return;
        if (marketSegment.isEmpty() || marketSegment.equalsIgnoreCase("NULL")) {
            context.getCounter(CancellationCounters.MISSING_MARKET_SEGMENT).increment(1);
        }
    }
}
```

```

        return;
    }
    segmentKey.set(marketSegment);
    context.write(segmentKey, ONE);
}
}

```

Reducer Class

```

import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class CancellationByMarketSegmentReducer
    extends Reducer<Text, IntWritable, Text, IntWritable> {
    @Override
    protected void reduce(Text key, Iterable<IntWritable> values, Context context)
        throws IOException, InterruptedException {
        int totalCancellations = 0;
        for (IntWritable value : values) totalCancellations += value.get();
        context.write(key, new IntWritable(totalCancellations));
    }
}

```

Driver Class

```

import java.io.IOException;
import java.util.*;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class CancellationByMarketSegmentDriver {
    public static class SortMapper extends Mapper<LongWritable, Text, NullWritable, Text> {
        @Override
        protected void map(LongWritable key, Text value, Context context)
            throws IOException, InterruptedException {
            String line = value.toString().trim();
            if (line.isEmpty()) return;
            String[] parts = line.split("\\t");
            if (parts.length < 2) return;
            try {
                String segment = parts[0].trim();
                int count = Integer.parseInt(parts[1].trim());
                context.write(NullWritable.get(), new Text(segment + "\\t" + count));
            } catch (NumberFormatException e) { }
        }
    }

    public static class SortReducer extends Reducer<NullWritable, Text, Text, IntWritable> {
        private static class SegmentCount {
            String segment; int count;
            SegmentCount(String segment, int count) { this.segment = segment; this.count = count; }
        }
        @Override
        protected void reduce(NullWritable key, Iterable<Text> values, Context context)
            throws IOException, InterruptedException {
            List<SegmentCount> results = new ArrayList<>();
            for (Text value : values) {
                String[] parts = value.toString().split("\\t");
                if (parts.length < 2) continue;
                try {
                    results.add(new SegmentCount(parts[0].trim(), Integer.parseInt(parts[1].trim())));
                } catch (NumberFormatException e) { }
            }
            results.sort((a,b) -> Integer.compare(b.count, a.count));
            for (SegmentCount item : results)
                context.write(new Text(item.segment), new IntWritable(item.count));
        }
    }

    public static void main(String[] args) throws Exception {
        if (args.length != 3) {
            System.err.println("Usage: CancellationByMarketSegmentDriver <input> <intermediate_output> <final_output>");
            System.exit(1);
        }
        Configuration conf = new Configuration();
        Job job1 = Job.getInstance(conf, "Hotel Booking - Cancellations by Market Segment");
        job1.setJarByClass(CancellationByMarketSegmentDriver.class);
        job1.setMapperClass(CancellationByMarketSegmentMapper.class);
        job1.setReducerClass(CancellationByMarketSegmentReducer.class);
        job1.setMapOutputKeyClass(Text.class);
    }
}

```

```

job1.setMapOutputValueClass(IntWritable.class);
job1.setOutputKeyClass(Text.class);
job1.setOutputValueClass(IntWritable.class);
FileInputFormat.addInputPath(job1, new Path(args[0]));
FileOutputFormat.setOutputPath(job1, new Path(args[1]));
if (!job1.waitForCompletion(true)) System.exit(1);

Job job2 = Job.getInstance(conf, "Hotel Booking - Sorted Cancellations by Market Segment");
job2.setJarByClass(CancellationByMarketSegmentDriver.class);
job2.setMapperClass(SortMapper.class);
job2.setReducerClass(SortReducer.class);
job2.setNumReduceTasks(1);
job2.setMapOutputKeyClass(NullWritable.class);
job2.setMapOutputValueClass(Text.class);
job2.setOutputKeyClass(Text.class);
job2.setOutputValueClass(IntWritable.class);
FileInputFormat.addInputPath(job2, new Path(args[1]));
FileOutputFormat.setOutputPath(job2, new Path(args[2]));
System.exit(job2.waitForCompletion(true) ? 0 : 1);
}
}

```

Execution

```

hadoop jar target\cancellation-market-segment-1.0.jar /user/DELL/hotel_project/input/Hotel_Booking_Clean.tsv
/user/DELL/hotel_project/output/cancellation_by_segment_counts
/user/DELL/hotel_project/output/cancellation_by_segment_sorted

```

```

004/
2026-09-15 20:04:51,654 INFO mapreduce.Job: Running job: job_1789479626440_0004
2026-09-15 20:05:05,929 INFO mapreduce.Job: Job job_1789479626440_0004 running in uber mode : false
2026-09-15 20:05:05,930 INFO mapreduce.Job: map 0% reduce 0%
2026-09-15 20:05:11,007 INFO mapreduce.Job: map 100% reduce 0%
2026-09-15 20:05:17,067 INFO mapreduce.Job: map 100% reduce 100%
2026-09-15 20:05:17,075 INFO mapreduce.Job: Job job_1789479626440_0004 completed successfully
2026-09-15 20:05:17,113 INFO mapreduce.Job: Counters: 54
File System Counters
FILE: Number of bytes read=137

```

MR2 execution completed successfully.

Result

```

C:\BDALFinalProject\CancellationByMarketSegment>hdfs dfs -cat /user/DELL/hotel_project/output/cancellation_by_segment_sorted/part-r
-00000
Online TA      20739
Groups 12097
Offline TA/TO  8311
Direct 1934
Corporate      992
Complementary  97
Aviation       52
Undefined      2

```

Cancellations by market segment sorted in descending order.

Market Segment	Canceled Bookings
Online TA	20,739
Groups	12,097
Offline TA/TO	8,311
Direct	1,934
Corporate	992
Complementary	97
Aviation	52
Undefined	2

Interpretation: Online TA had the highest number of canceled bookings with 20,739 cancellations, followed by Groups with 12,097 and Offline TA/TO with 8,311.

5.3 Top 10 Countries by Total Stayed Nights

This analysis considers only non-canceled bookings (`is_canceled = 0`). For each record, total stayed nights are calculated as `stays_in_weekend_nights + stays_in_week_nights`. The first job sums stayed nights by country and the second job returns the global Top 10 countries.

Mapper Class

```
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class Top10CountriesByStayedNightsMapper
    extends Mapper<LongWritable, Text, Text, IntWritable> {
    private final Text countryKey = new Text();
    private final IntWritable nightsValue = new IntWritable();
    public enum StayedNightCounters { MALFORMED_ROWS, MISSING_COUNTRY, INVALID_NUMERIC_VALUE }

    @Override
    protected void map(LongWritable key, Text value, Context context)
        throws IOException, InterruptedException {
        String[] fields = value.toString().split("\\t", -1);
        if (fields.length < 17) {
            context.getCounter(StayedNightCounters.MALFORMED_ROWS).increment(1);
            return;
        }
        try {
            int isCanceled = Integer.parseInt(fields[1].trim());
            if (isCanceled != 0) return;
            int weekendNights = Integer.parseInt(fields[5].trim());
            int weekNights = Integer.parseInt(fields[6].trim());
            String country = fields[8].trim();
            if (country.isEmpty() || country.equalsIgnoreCase("UNKNOWN") || country.equalsIgnoreCase("NULL")) {
                context.getCounter(StayedNightCounters.MISSING_COUNTRY).increment(1);
                return;
            }
            int totalNights = weekendNights + weekNights;
            countryKey.set(country);
            nightsValue.set(totalNights);
            context.write(countryKey, nightsValue);
        } catch (NumberFormatException e) {
            context.getCounter(StayedNightCounters.INVALID_NUMERIC_VALUE).increment(1);
        }
    }
}
```

Reducer Class

```
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class Top10CountriesByStayedNightsReducer
    extends Reducer<Text, IntWritable, Text, IntWritable> {
    @Override
    protected void reduce(Text key, Iterable<IntWritable> values, Context context)
        throws IOException, InterruptedException {
        int totalStayedNights = 0;
        for (IntWritable value : values) totalStayedNights += value.get();
        context.write(key, new IntWritable(totalStayedNights));
    }
}
```

Driver Class

```
import java.io.IOException;
import java.util.*;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class Top10CountriesByStayedNightsDriver {
    public static class Top10Mapper extends Mapper<LongWritable, Text, NullWritable, Text> {
        @Override
```

```

protected void map(LongWritable key, Text value, Context context)
    throws IOException, InterruptedException {
    String line = value.toString().trim();
    if (line.isEmpty()) return;
    String[] parts = line.split("\\t");
    if (parts.length < 2) return;
    try {
        context.write(NullWritable.get(), new Text(parts[0].trim() + "\\t" +
            Integer.parseInt(parts[1].trim())));
    } catch (NumberFormatException e) { }
}

public static class Top10Reducer extends Reducer<NullWritable, Text, Text, IntWritable> {
    private static class CountryNights {
        String country; int nights;
        CountryNights(String country, int nights) { this.country = country; this.nights = nights; }
    }
    @Override
    protected void reduce(NullWritable key, Iterable<Text> values, Context context)
        throws IOException, InterruptedException {
        PriorityQueue<CountryNights> top10 = new PriorityQueue<>(10,
            Comparator.comparingInt(a -> a.nights));
        for (Text value : values) {
            String[] parts = value.toString().split("\\t");
            if (parts.length < 2) continue;
            try {
                CountryNights cn = new CountryNights(parts[0].trim(),
                    Integer.parseInt(parts[1].trim()));
                top10.add(cn);
                if (top10.size() > 10) top10.poll();
            } catch (NumberFormatException e) { }
        }
        List<CountryNights> result = new ArrayList<>(top10);
        result.sort((a,b) -> Integer.compare(b.nights, a.nights));
        for (CountryNights item : result)
            context.write(new Text(item.country), new IntWritable(item.nights));
    }
}

public static void main(String[] args) throws Exception {
    if (args.length != 3) {
        System.err.println("Usage: Top10CountriesByStayedNightsDriver <input> <intermediate_output> <final_output>");
        System.exit(1);
    }
    Configuration conf = new Configuration();
    Job job1 = Job.getInstance(conf, "Hotel Booking - Total Stayed Nights by Country");
    job1.setJarByClass(Top10CountriesByStayedNightsDriver.class);
    job1.setMapperClass(Top10CountriesByStayedNightsMapper.class);
    job1.setReducerClass(Top10CountriesByStayedNightsReducer.class);
    job1.setMapOutputKeyClass(Text.class);
    job1.setMapOutputValueClass(IntWritable.class);
    job1.setOutputKeyClass(Text.class);
    job1.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job1, new Path(args[0]));
    FileOutputFormat.setOutputPath(job1, new Path(args[1]));
    if (!job1.waitForCompletion(true)) System.exit(1);

    Job job2 = Job.getInstance(conf, "Hotel Booking - Top 10 Countries by Stayed Nights");
    job2.setJarByClass(Top10CountriesByStayedNightsDriver.class);
    job2.setMapperClass(Top10Mapper.class);
    job2.setReducerClass(Top10Reducer.class);
    job2.setNumReduceTasks(1);
    job2.setMapOutputKeyClass(NullWritable.class);
    job2.setMapOutputValueClass(Text.class);
    job2.setOutputKeyClass(Text.class);
    job2.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job2, new Path(args[1]));
    FileOutputFormat.setOutputPath(job2, new Path(args[2]));
    System.exit(job2.waitForCompletion(true) ? 0 : 1);
}
}

```

Execution

hadoop jar target\top10-countries-stayed-nights-1.0.jar /user/DELL/hotel_project/input/Hotel_Booking_Clean.tsv
 /user/DELL/hotel_project/output/stayed_nights_by_country
 /user/DELL/hotel_project/output/top10_countries_stayed_nights

```

0007
2026-09-15 20:13:41,918 INFO mapreduce.Job: Running job: job_1789479626440_0006
2026-09-15 20:13:55,135 INFO mapreduce.Job: Job job_1789479626440_0006 running in uber mode : false
2026-09-15 20:13:55,135 INFO mapreduce.Job: map 0% reduce 0%
2026-09-15 20:14:00,220 INFO mapreduce.Job: map 100% reduce 0%
2026-09-15 20:14:05,286 INFO mapreduce.Job: map 100% reduce 100%
2026-09-15 20:14:05,293 INFO mapreduce.Job: Job job_1789479626440_0006 completed successfully
2026-09-15 20:14:05,328 INFO mapreduce.Job: Counters: 54
File System Counters

```

MR3 execution completed successfully.

Result

```
C:\BDALFinalProject\Top10CountriesByStayedNights>hdfs dfs -cat /user/DELL/hotel_project/output/top10_countries_stayed_nights/part-r-00000
PRT      52857
GBR      48181
FRA      29315
DEU      21154
ESP      19024
IRL      13396
ITA      7737
BEL      6852
NLD      6317
CHE      4591
```

Top 10 countries by total stayed nights.

Country	Total Stayed Nights
PRT	52,857
GBR	48,181
FRA	29,315
DEU	21,154
ESP	19,024
IRL	13,396
ITA	7,737
BEL	6,852
NLD	6,317
CHE	4,591

Interpretation: Portugal (PRT) recorded the highest total stayed nights with 52,857 nights among non-canceled bookings, followed by the United Kingdom (GBR) with 48,181 and France (FRA) with 29,315.

6. Pig Hotel Booking Data Analysis

Apache Pig was used to perform five analyses on the same HDFS TSV dataset. PigStorage(\t) loads the 17-field cleaned schema, and the scripts use FILTER, GROUP, FOREACH, COUNT, AVG, ORDER, LIMIT and STORE as appropriate.

6.1 Top 5 Countries by Booking Count

Returns the five countries containing the largest number of valid booking records. This operation provides a Pig-based cross-check of the leading country counts from MapReduce MR1.

Script

```
data = LOAD '/user/DELL/hotel_project/input/Hotel_Booking_Clean.tsv'
USING PigStorage('\t')
AS (
  hotel:chararray,
  is_canceled:int,
  lead_time:int,
  arrival_date_year:int,
  arrival_date_month:chararray,
  stays_in_weekend_nights:int,
  stays_in_week_nights:int,
  adults:int,
  country:chararray,
  market_segment:chararray,
  distribution_channel:chararray,
  is_repeated_guest:int,
  deposit_type:chararray,
  customer_type:chararray,
```

```

    adr:double,
    reservation_status:chararray,
    reservation_status_date:chararray
);

valid = FILTER data BY country IS NOT NULL
    AND country != "
    AND country != 'UNKNOWN';
grouped = GROUP valid BY country;
counts = FOREACH grouped GENERATE group AS country, COUNT(valid) AS booking_count;
sorted = ORDER counts BY booking_count DESC;
top5 = LIMIT sorted 5;
STORE top5 INTO '/user/DELL/hotel_project/pig_output/top5_countries'
USING PigStorage('\t');

```

Execution

`pig -x mapreduce scripts\top5Countries.pig`

```

onStatus=SUCCEEDED. Redirecting to job history server
2026-09-15 20:50:43,420 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - 100% complete
2026-09-15 20:50:43,441 [main] INFO org.apache.pig.tools.pigstats.mapreduce.SimplePigStats - Script Statistics:

HadoopVersion  PigVersion  UserId  StartedAt  FinishedAt  Features
3.2.4  0.18.0  DELL  2026-09-15 20:48:52  2026-09-15 20:50:43  GROUP_BY,ORDER_BY,FILTER,LIMIT

Success!

Job Stats (time in seconds):
JobId  Maps  Reduces  MaxMapTime  MinMapTime  AvgMapTime  MedianMapTime  MaxReduceTime  MinReduceTime  AvgReduceTime
e  MedianReduceTime  Alias  Feature  Outputs
job_1789479626440_0007  1  1  7  7  7  7  4  4  4  4  counts,data,grouped,valid
GROUP_BY,COMBINER
job_1789479626440_0008  1  1  3  3  3  3  3  3  3  3  sorted SAMPLER
job_1789479626440_0009  1  1  3  3  3  3  4  4  4  4  sorted ORDER_BY,COMBINER

```

Pig Operation 1 execution completed successfully.

Result

```

C:\BDALFinalProject\PigAnalysis>hdfs dfs -cat /user/DELL/hotel_project/pig_output/top5_countries/part-r-00000
PRT 48590
GBR 12129
FRA 10415
ESP 8568
DEU 7287

C:\BDALFinalProject\PigAnalysis>

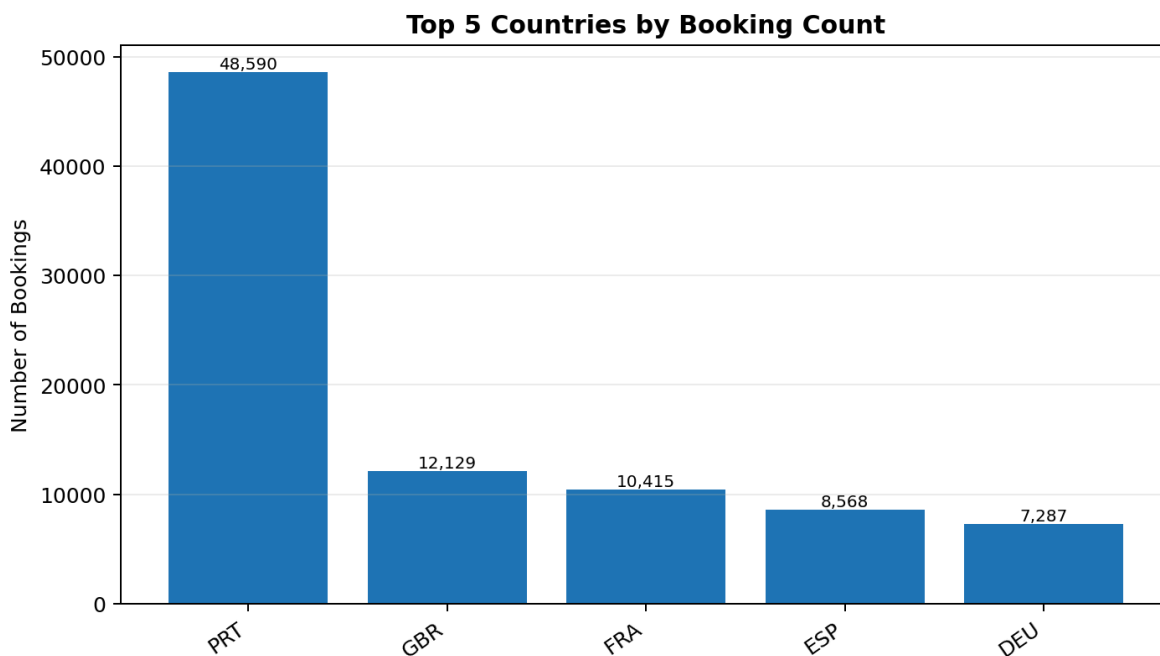
```

Top 5 countries by booking count.

Country	Booking Count
PRT	48,590
GBR	12,129
FRA	10,415
ESP	8,568
DEU	7,287

Interpretation: PRT, GBR, FRA, ESP and DEU were the top five countries by booking count. The values match the leading results from the MapReduce country-count analysis.

Visualization



Top 5 countries by booking count.

6.2 Top 10 Countries by Average ADR

ADR means Average Daily Rate. The script considers valid non-canceled bookings with ADR greater than zero, calculates country-level average ADR, sorts descending, and returns the top ten countries.

Script

```
data = LOAD '/user/DELL/hotel_project/input/Hotel_Booking_Clean.tsv'
USING PigStorage('\t')
AS (
  hotel:chararray,
  is_canceled:int,
  lead_time:int,
  arrival_date_year:int,
  arrival_date_month:chararray,
  stays_in_weekend_nights:int,
  stays_in_week_nights:int,
  adults:int,
  country:chararray,
  market_segment:chararray,
  distribution_channel:chararray,
  is_repeated_guest:int,
  deposit_type:chararray,
  customer_type:chararray,
  adr:double,
  reservation_status:chararray,
  reservation_status_date:chararray
);

valid = FILTER data BY country IS NOT NULL
AND country != ''
AND country != 'UNKNOWN'
AND is_canceled == 0
AND adr IS NOT NULL
AND adr > 0.0;

grouped = GROUP valid BY country;
averages = FOREACH grouped GENERATE group AS country, AVG(valid.adr) AS average_adr;
sorted = ORDER averages BY average_adr DESC;
top10 = LIMIT sorted 10;
STORE top10 INTO '/user/DELL/hotel_project/pig_output/top10_countries_average_adr'
USING PigStorage('\t');
```

Execution

```
pig -x mapreduce scripts\top10CountriesByAverageADR.pig
```

```

onStatus=SUCCEEDED. Redirecting to job history server
2026-09-15 20:56:24,301 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - 100% complete
2026-09-15 20:56:24,321 [main] INFO org.apache.pig.tools.pigstats.mapreduce.SimplePigStats - Script Statistics:

HadoopVersion  PigVersion  UserId  StartedAt  FinishedAt  Features
3.2.4  0.18.0  DELL  2026-09-15 20:54:38  2026-09-15 20:56:24  GROUP_BY,ORDER_BY,FILTER,LIMIT

Success!

Job Stats (time in seconds):
JobId  Maps  Reduces  MaxMapTime  MinMapTime  AvgMapTime  MedianMapTime  MaxReduceTime  MinReduceTime  AvgReduceTime
e  MedianReduceTime  Alias  Feature  Outputs
job_1789479626440_0011  1  1  5  5  5  4  4  4  averages,data,grouped,valid
GROUP_BY,COMBINER
job_1789479626440_0012  1  1  3  3  3  3  3  3  sorted SAMPLER
job_1789479626440_0013  1  1  3  3  3  3  3  3  sorted ORDER_BY,COMBINER

job_1789479626440_0014  1  1  3  3  3  3  3  3  sorted /user/DELL/h
otel_project/pig_output/top10_countries_average_adr,

```

Pig Operation 2 execution completed successfully.

Result

```

C:\BDALFinalProject\PigAnalysis>hdfs dfs -cat /user/DELL/hotel_project/pig_output/top10_countries_average_adr/part-r-00000
AND 295.0
DJI 273.0
AIA 265.0
FRO 202.5
LAO 181.66500000000002
BGD 175.76333333333332
NCL 175.5
COM 165.305
GEO 163.23857142857145
TMP 160.15

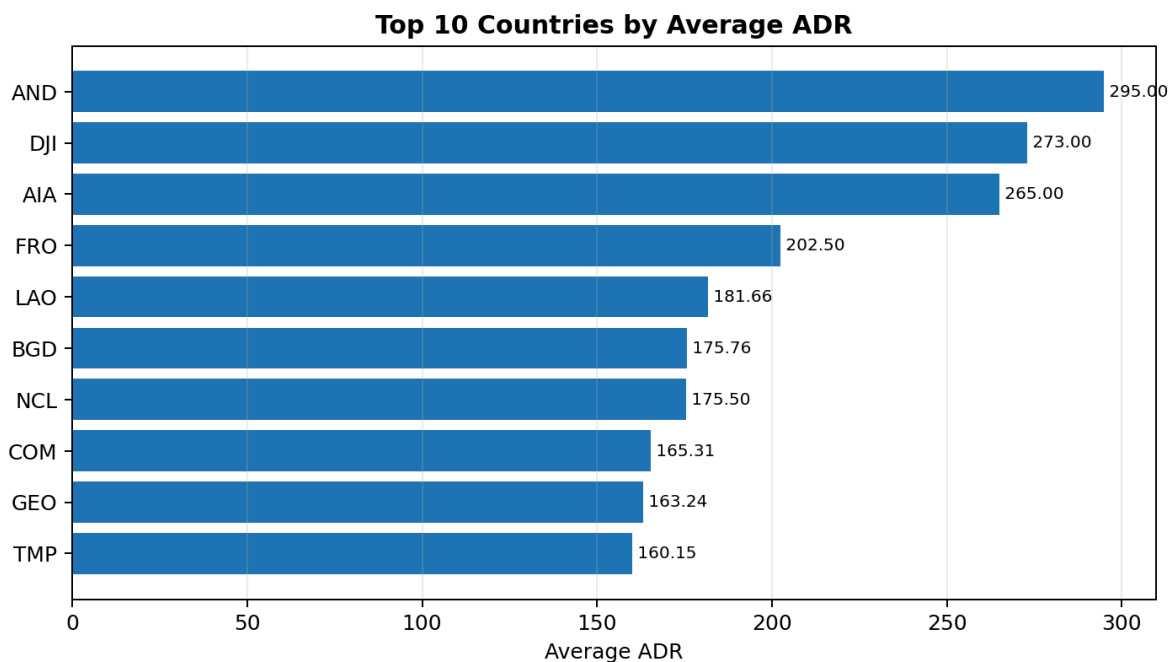
```

Top 10 countries by average ADR.

Country	Average ADR
AND	295.00
DJI	273.00
AIA	265.00
FRO	202.50
LAO	181.67
BGD	175.76
NCL	175.50
COM	165.31
GEO	163.24
TMP	160.15

Interpretation: Andorra (AND) had the highest average ADR at 295.00 among the filtered non-canceled bookings, followed by Djibouti (DJI) at 273.00 and Anguilla (AIA) at 265.00. Average ADR should be interpreted with booking volume in mind because some countries may have few observations.

Visualization



Top 10 countries by average ADR.

6.3 Top 10 Countries by Number of Cancellations

Filters canceled bookings, groups by country, counts the canceled records for each country, and returns the ten highest cancellation counts.

Script

```
data = LOAD '/user/DELL/hotel_project/input/Hotel_Booking_Clean.tsv'
USING PigStorage('\t')
AS (
  hotel:chararray,
  is_canceled:int,
  lead_time:int,
  arrival_date_year:int,
  arrival_date_month:chararray,
  stays_in_weekend_nights:int,
  stays_in_week_nights:int,
  adults:int,
  country:chararray,
  market_segment:chararray,
  distribution_channel:chararray,
  is_repeated_guest:int,
  deposit_type:chararray,
  customer_type:chararray,
  adr:double,
  reservation_status:chararray,
  reservation_status_date:chararray
);

valid = FILTER data BY country IS NOT NULL
      AND country != ''
      AND country != 'UNKNOWN'
      AND is_canceled == 1;
grouped = GROUP valid BY country;
counts = FOREACH grouped GENERATE group AS country, COUNT(valid) AS cancellation_count;
sorted = ORDER counts BY cancellation_count DESC;
top10 = LIMIT sorted 10;
STORE top10 INTO '/user/DELL/hotel_project/pig_output/top10_countries_cancellations'
USING PigStorage('\t');
```

Execution

```
pig -x mapreduce scripts\top10CountriesByCancellations.pig
```

```

2026-09-15 21:01:45,416 [main] INFO org.apache.hadoop.mapred.ClientServiceDelegate - Application state is completed. FinalApplicationStatus=SUCCEEDED. Redirecting to job history server
2026-09-15 21:01:45,438 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - 100% complete
2026-09-15 21:01:45,459 [main] INFO org.apache.pig.tools.pigstats.mapreduce.SimplePigStats - Script Statistics:

HadoopVersion  PigVersion      UserId  StartedAt       FinishedAt       Features
3.2.4    0.18.0  DELL    2026-09-15 21:00:04  2026-09-15 21:01:45  GROUP_BY,ORDER_BY,FILTER,LIMIT

Success!

Job Stats (time in seconds):
JobId  Maps    Reduces  MaxMapTime    MinMapTime    AvgMapTime    MedianMapTime    MaxReduceTime    MinReduceTime    AvgReduceTime
e      MedianReducetime    Alias    Feature Outputs
job_1789479626440_0015  1      1      4      4      4      4      3      3      counts,data,grouped,valid
GROUP_BY,COMBINER
job_1789479626440_0016  1      1      3      3      3      3      3      3      sorted SAMPLER
job_1789479626440_0017  1      1      3      3      3      4      4      4      sorted ORDER_BY,COMBINER
job_1789479626440_0018  1      1      3      3      3      3      3      3      sorted /user/DELL/h
otel_project/pig_output/top10_countries_cancellations

```

Pig Operation 3 execution completed successfully.

Result

```

C:\BDALFinalProject\PigAnalysis>hdfs dfs -cat /user/DELL/hotel_project/pig_output/top10_countries_cancellations/part-r-00000
PRT    27519
GBR    2453
ESP    2177
FRA    1934
ITA    1333
DEU    1218
IRL    832
BRA    830
USA    501
BEL    474

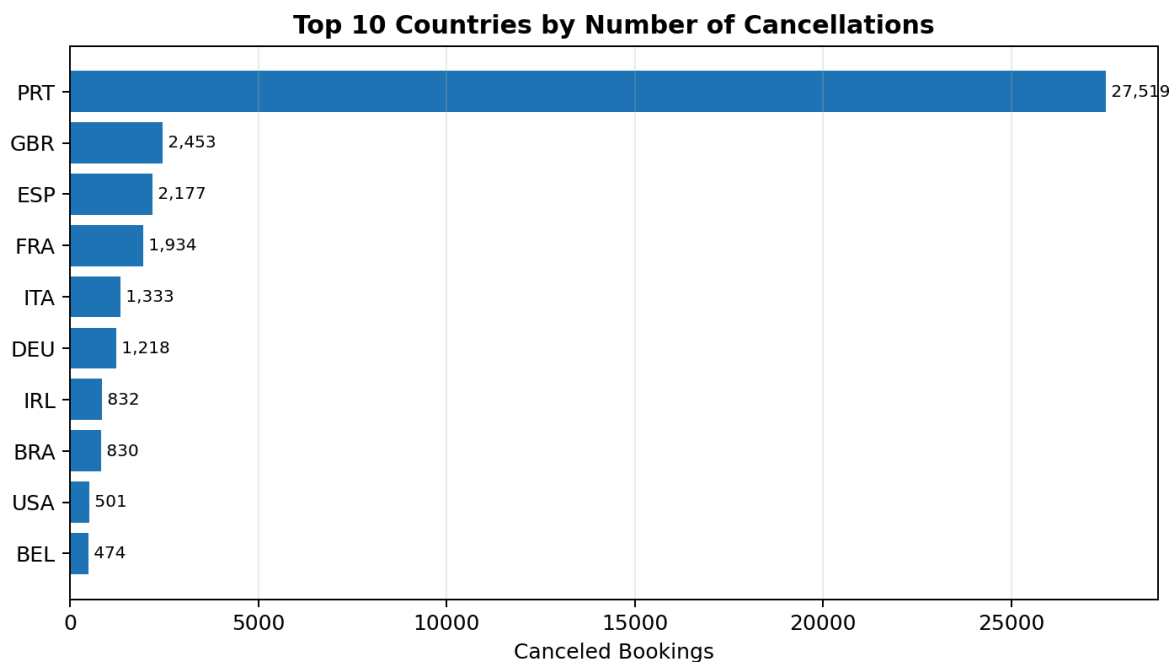
```

Top 10 countries by number of cancellations.

Country	Canceled Bookings
PRT	27,519
GBR	2,453
ESP	2,177
FRA	1,934
ITA	1,333
DEU	1,218
IRL	832
BRA	830
USA	501
BEL	474

Interpretation: Portugal (PRT) had the highest number of canceled bookings with 27,519 cancellations, followed by the United Kingdom (GBR) with 2,453 and Spain (ESP) with 2,177.

Visualization



Top 10 countries by number of cancellations.

6.4 Market Segments by Booking Count

Counts bookings for each market segment and sorts the available segments in descending order. The dataset contains eight observed market-segment values, so LIMIT 10 naturally returns all eight.

Script

```
data = LOAD '/user/DELL/hotel_project/input/Hotel_Booking_Clean.tsv'
USING PigStorage('\t')
AS (
  hotel:chararray,
  is_canceled:int,
  lead_time:int,
  arrival_date_year:int,
  arrival_date_month:chararray,
  stays_in_weekend_nights:int,
  stays_in_week_nights:int,
  adults:int,
  country:chararray,
  market_segment:chararray,
  distribution_channel:chararray,
  is_repeated_guest:int,
  deposit_type:chararray,
  customer_type:chararray,
  adr:double,
  reservation_status:chararray,
  reservation_status_date:chararray
);

valid = FILTER data BY market_segment IS NOT NULL
      AND market_segment != "";
grouped = GROUP valid BY market_segment;
counts = FOREACH grouped GENERATE group AS market_segment, COUNT(valid) AS booking_count;
sorted = ORDER counts BY booking_count DESC;
top10 = LIMIT sorted 10;
STORE top10 INTO '/user/DELL/hotel_project/pig_output/top10_market_segments'
USING PigStorage('\t');
```

Execution

```
pig -x mapreduce scripts\top10MarketSegments.pig
```

```

CCEDEED. Redirecting to job history server
2026-09-15 21:07:24,718 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - 100% complete
2026-09-15 21:07:24,753 [main] INFO org.apache.pig.tools.pigstats.mapreduce.SimplePigStats - Script Statistics:

HadoopVersion  PigVersion  UserId  StartedAt  FinishedAt  Features
3.2.4  0.18.0  DELL  2026-09-15 21:05:37  2026-09-15 21:07:24  GROUP_BY,ORDER_BY,FILTER,LIMIT

Success!

Job Stats (time in seconds):
JobId  Maps  Reduces  MaxMapTime  MinMapTime  AvgMapTime  MedianMapTime  MaxReduceTime  MinReduceTime  AvgReduceTime  MedianR
educetime  Alias  Feature  Outputs
job_1789479626440_0019  1  1  5  5  5  5  3  3  3  3  counts,data,grouped,valid  GROUP_B
Y,COMBINER
job_1789479626440_0020  1  1  3  3  3  3  3  3  3  3  sorted SAMPLER
job_1789479626440_0021  1  1  3  3  3  3  3  3  3  3  sorted ORDER_BY,COMBINER
job_1789479626440_0022  1  1  3  3  3  3  3  3  3  3  sorted /user/DELL/hotel_projec
t/pig_output/top10_market_segments,

```

Pig Operation 4 execution completed successfully.

Result

```

C:\BDALFinalProject\PigAnalysis>hdfs dfs -cat /user/DELL/hotel_project/pig_output/top10_market_segments/part-r-00000
Online TA 56477
Offline TA/TO 24219
Groups 19811
Direct 12606
Corporate 5295
Complementary 743
Aviation 237
Undefined 2

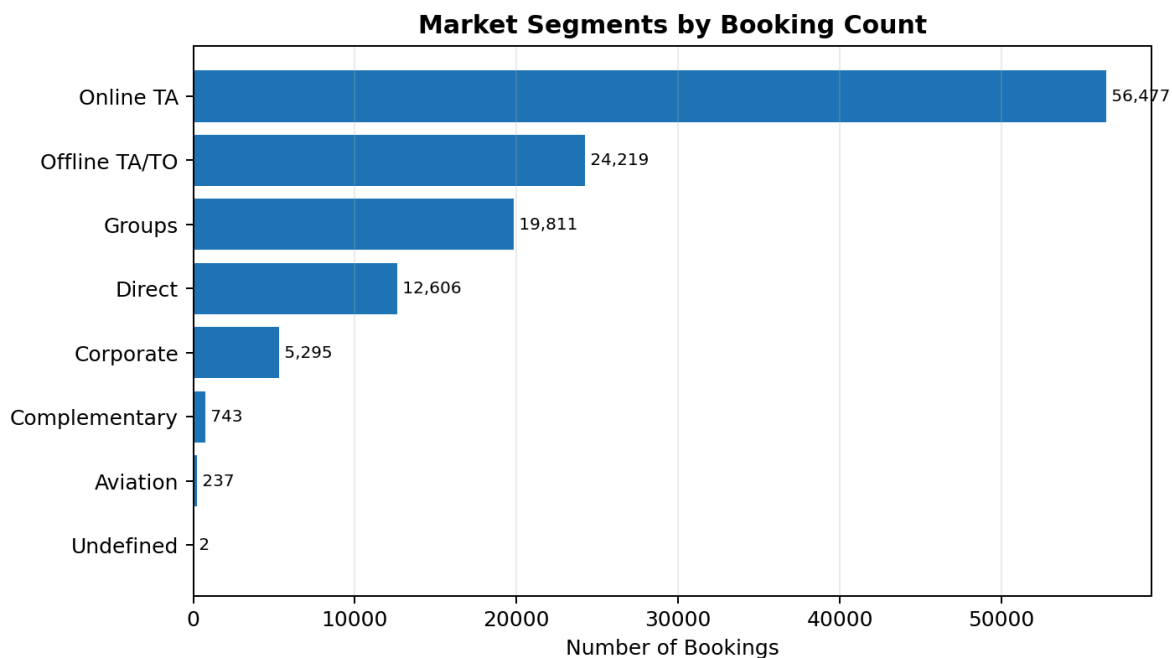
```

Market segments by booking count.

Market Segment	Booking Count
Online TA	56,477
Offline TA/TO	24,219
Groups	19,811
Direct	12,606
Corporate	5,295
Complementary	743
Aviation	237
Undefined	2

Interpretation: Online TA was the largest market segment with 56,477 booking records, followed by Offline TA/TO with 24,219 and Groups with 19,811.

Visualization



Market segments by booking count.

6.5 Top 10 Arrival Months by Booking Count

Groups booking records by arrival month, counts the records for each month, sorts the counts in descending order and returns the ten highest months.

Script

```
data = LOAD '/user/DELL/hotel_project/input/Hotel_Booking_Clean.tsv'
USING PigStorage('\t')
AS (
  hotel:chararray,
  is_canceled:int,
  lead_time:int,
  arrival_date_year:int,
  arrival_date_month:chararray,
  stays_in_weekend_nights:int,
  stays_in_week_nights:int,
  adults:int,
  country:chararray,
  market_segment:chararray,
  distribution_channel:chararray,
  is_repeated_guest:int,
  deposit_type:chararray,
  customer_type:chararray,
  adr:double,
  reservation_status:chararray,
  reservation_status_date:chararray
);

valid = FILTER data BY arrival_date_month IS NOT NULL
      AND arrival_date_month != "";
grouped = GROUP valid BY arrival_date_month;
counts = FOREACH grouped GENERATE group AS arrival_month, COUNT(valid) AS booking_count;
sorted = ORDER counts BY booking_count DESC;
top10 = LIMIT sorted 10;
STORE top10 INTO '/user/DELL/hotel_project/pig_output/top10_arrival_months'
USING PigStorage('\t');
```

Execution

```
pig -x mapreduce scripts\top10ArrivalMonths.pig
```

```

2026-09-15 21:12:32,503 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - 100% complete
2026-09-15 21:12:32,532 [main] INFO org.apache.pig.tools.pigstats.mapreduce.SimplePigStats - Script Statistics:

HadoopVersion PigVersion UserId StartedAt FinishedAt Features
3.2.4 0.18.0 DELL 2026-09-15 21:10:42 2026-09-15 21:12:32 GROUP_BY,ORDER_BY,FILTER,LIMIT

Success!

Job Stats (time in seconds):
JobId Maps Reduces MaxMapTime MinMapTime AvgMapTime MedianMapTime MaxReduceTime MinReduceTime AvgReduceTime MedianR
educetime Alias Feature Outputs
job_1789479626440_0023 1 1 5 5 5 5 4 4 4 4 counts,data,grouped,valid GROUP_B
Y,COMBINER
job_1789479626440_0024 1 1 3 3 3 3 3 3 3 3 sorted SAMPLER
job_1789479626440_0025 1 1 3 3 3 3 3 3 3 3 sorted ORDER_BY,COMBINER
job_1789479626440_0026 1 1 3 3 3 3 3 3 3 3 sorted /user/DELL/hotel_project
t/pig_output/top10_arrival_months,

Input(s):
Successfully read 119390 records (12588684 bytes) from: "/user/DELL/hotel_project/input/Hotel_Booking_Clean.tsv"

```

Pig Operation 5 execution completed successfully.

Result

```

C:\BDALFinalProject\PigAnalysis>hdfs dfs -cat /user/DELL/hotel_project/pig_output/top10_arrival_months/part-r-00000
August 13877
July 12661
May 11791
October 11160
April 11089
June 10939
September 10508
March 9794
February 8068
November 6794

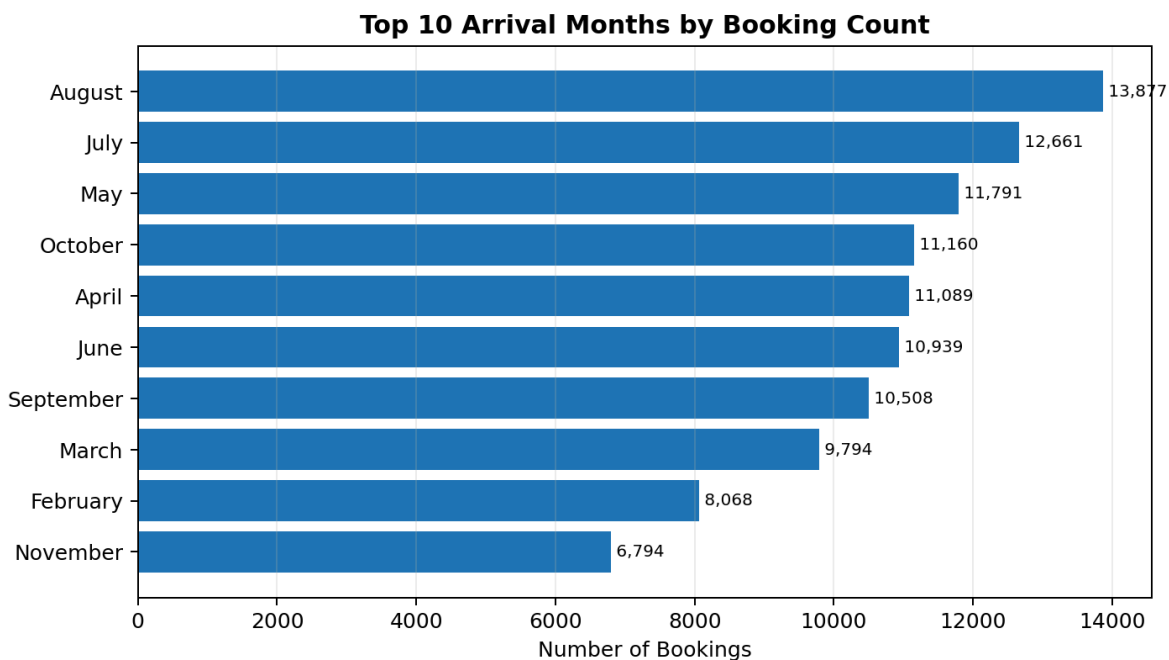
```

Top 10 arrival months by booking count.

Arrival Month	Booking Count
August	13,877
July	12,661
May	11,791
October	11,160
April	11,089
June	10,939
September	10,508
March	9,794
February	8,068
November	6,794

Interpretation: August had the highest booking count with 13,877 records, followed by July with 12,661 and May with 11,791. Because this operation ranks total records across the complete dataset, the result describes booking volume rather than a causal seasonal effect.

Visualization



Top 10 arrival months by booking count.

7. GitHub Repository

The completed Hotel Booking Hadoop project was published to a public GitHub repository so that the source code, Pig scripts, datasets, output files, execution screenshots, and project documentation can be reviewed from one location.

Repository URL: <https://github.com/25tarek/Hotel-Booking-Data-Analysis-using-Hadoop>

Repository Structure and Contents

- **Preprocessing/** - Java preprocessing source and Maven configuration.
- **Top10Countries/** - MapReduce MR1 source code and Top 10 country output.
- **CancellationByMarketSegment/** - MapReduce MR2 source code and cancellation output.
- **Top10CountriesByStayedNights/** - MapReduce MR3 source code and stayed-night output.
- **PigAnalysis/** - Five Pig scripts and their saved output files.
- **data/** - Raw CSV and processed TSV datasets.
- **screenshots/** - Execution and result screenshots used as project evidence.
- **README.md** - Project overview, tools, operations, and key results.



GitHub repository containing the project source, data folders, outputs, screenshots, and README.

8. Conclusion

This project demonstrates a complete Hadoop-based workflow for Hotel Booking data analysis. The raw `hotel_bookings.csv` file contained 119,390 booking records. Java and Apache Commons CSV were used to create a clean 17-field TSV dataset without losing valid rows, and the cleaned file was stored in HDFS. Three Java MapReduce analyses and five Apache Pig analyses were then executed successfully.

The MapReduce results showed that Portugal (PRT) had the highest booking count with 48,590 records and the highest total stayed nights with 52,857 nights among non-canceled bookings. Online TA was the market segment with the largest number of cancellations, totaling 20,739. The Pig analyses independently confirmed the leading country booking counts, identified Andorra (AND) as the highest average-ADR country within the applied non-canceled/positive-ADR filter, showed Portugal with 27,519 canceled bookings, identified Online TA as the largest market segment with 56,477 bookings, and found August as the highest-volume arrival month with 13,877 bookings.

Overall, the outputs demonstrate how Hadoop MapReduce can provide explicit low-level distributed aggregation and ranking logic, while Apache Pig can express comparable grouping, filtering and ranking analyses with shorter high-level scripts. The same workflow can be extended to larger hotel-booking datasets, additional years, or more detailed analyses such as cancellation rates, customer segmentation and revenue-oriented measures. The complete project artifacts were also organized and published in the public GitHub repository listed in Section 7, providing a consolidated record of the source code, scripts, outputs, datasets, screenshots, and project documentation.

9. References

1. Hotel Booking Demand dataset, Kaggle: <https://www.kaggle.com/datasets/jessemostipak/hotel-booking-demand>
2. Apache Hadoop 3.2.4 Documentation: <https://hadoop.apache.org/docs/r3.2.4/>
3. Apache Pig 0.18.0 Documentation: <https://pig.apache.org/docs/r0.18.0/>
4. Project repository, GitHub: <https://github.com/25tarek/Hotel-Booking-Data-Analysis-using-Hadoop>